

本文档为 BitFS 设计文档体系的第四层：测试用例设计。

文档体系：(总体设计) 1. 概念设计 — 项目愿景、核心概念、架构概览 2. 系统设计 — 模块划分、接口定义、数据流 3. 详细设计 — 算法、数据结构、协议细节 4. 测试用例（本文档） — 测试用例设计

设计章节交叉引用： 系统设计章节（如” 第二节” ）见 [2-SystemDesign](#)，详细设计章节（如” 第四-B节” ）见 [3-DetailedDesign](#)。

BitFS 测试用例设计

本章定义完整的测试用例规范，作为系统设计的组成部分。每个测试用例可追溯到设计章节，并直接对应代码实现。测试用例在系统设计阶段完成定义，先于实现代码。

设计原则

- 1. 三段式格式: 每个测试用例采用” 前置条件 → 操作 → 期望结果”
- 2. 标签分类: [unit] 单元测试 | [edge] 边界条件 | [integration] 集成测试 | [property] 属性不变量 | [security] 安全性
- 3. 覆盖率目标: 每个 package ≥80% 行覆盖率
- 4. 命名映射: 规范 ID T{N}. {M}. {K} → 代码函数名 Test{Category}_{Scenario}
- 5. 独立性: 每个测试可独立运行，无测试间依赖
- 6. 确定性: 所有测试使用固定种子/mock，禁止随机行为
- 7. 表驱动: Go 惯例，使用 []struct{ name string; ... } + t.Run()
- 8. 断言框架: testify/require (致命错误) 与 testify/assert (非致命错误) 区分使用

总览

#	类别	设计参照	代码文件	现有	目标
T1	Protobuf Schema	四	proto/ bitfs_test.go	61	72
T2	HD Wallet	二-B		33	44

#	类别	设计参照	代码文件	现有	目标
			method42/ hdwallet_test.go		
T3	Method 42 加密	五, 二-B.D	method42/ encrypt_test.go	23	40
T4	Vault 管理	二	method42/ vault_test.go	24	32
T5	握手协议	十三-B.B	method42/ handshake_test.go	8	17
T6	Key Capsule & HTLC	十三-B.D	method42/ keycapsule_test.go	9	18
T7	Key Cache	十一	method42/ keycache_test.go	8	12
T8	Metanet 节 点与构建器	四-B	metanet/ metanet_test.go	34	49
T9	文件系统操 作 (14种)	四-B, 三	metanet/fs_test.go	46	73
T10	Metanet 解 析器	四-B	metanet/ metanet_test.go	10	19
T11	内容寻址存 储	七	storage/ store_test.go	22	26
T12	SPV 客户端	七	spv/spv_test.go	41	50
T13	DNSLink	六	dnslink/ dnslink_test.go	4	13
T14	URI 寻址	六	addressing/ uri_test.go	11	14
T15	Daemon HTTP	十三-B.A	daemon/ daemon_test.go	99	111
T16	CLI 工具	八, 九-B	cmd/*_test.go	139	181

#	类别	设计参照	代码文件	现有	目标
T17	CLI 通用	八	cli/common_test.go	16	22
T18	配置与错误处理	十六	config/ config_test.go	33	38
T19	Shell 交互	十, 九-B.C	shell/ shell_test.go	21	28
T20	集成/端到端	全部	integration/ e2e_test.go	7	27
T21	属性不变量	多处	(新建)	0	23
T22	安全性	五, 十一, 十三-B	(新建)	0	17
T23	会话管理 (Lock/ Unlock)	二十一	method42/ session_test.go	0	24
T24	权限管理 (ACL + 群签名)	二十二	acl/acl_test.go	0	10
T25	收益权/ISO	十一-f	revenue/ iso_test.go	0	8
T26	同步与批量发布 (bsync/ bput)	二十三	sync/bsync_test.go	0	6
T27	Paymail 集成	六, 十六-B	paymail/ paymail_test.go	0	6
	合计			649	~980

注: T8 与 T10 共享 metanet/metanet_test.go, T8 覆盖节点构建器测试, T10 覆盖路径解析器测试。

T1. Protobuf Schema

设计参照: 第四节 | 代码文件: `src/proto/bitfs_test.go` | 现有/目标: 61 / 72

子类别	说明	现有	新增
T1.1 BitFSPayload 字段验证	所有字段的序列化/反序列化, 包括零值与默认值	18	+3
T1.2 ChildEntry 结构	目录项的完整字段验证, 含 LinkType 枚举	12	+2
T1.3 枚举值	NodeType/ OpType/ AccessLevel/ LinkType 所有枚举值	15	+2
T1.4 版本兼容性	新增字段的向前/向后兼容, unknown 字段保留	8	+2
T1.5 边界值	极大 file_size、空 keywords、超长 metadata	8	+2

T2. HD Wallet

设计参照: 第二-B节 | 代码文件: `src/internal/method42/hdwallet_test.go` | 现有/目标: 33 / 44

子类别	说明	现有	新增
T2.1 BIP39 助记词	12/24 词生成、语言校验、checksum 验证	8	+2
T2.2 BIP32 派生路径	m/44' /236' / account' /	10	+3

子类别	说明	现有	新增
	change/index 完整路径表		
T2.3 文件系统→HD 映射	路径 /a/b/c → HD 路径的确定性映射 (见二-B.C)	6	+2
T2.4 Vault 隔离	不同 account 派生的密钥互不相关	5	+2
T2.5 恢复确定性	助记词 → seed → 完整密钥树的确定性重建	4	+2

T3. Method 42 加密 ★

设计参照: 第五节, 第二-B.D节 代码文件: `src/internal/method42/encrypt_test.go` 测试函数数: 现有 23 / 目标 40

T3.1: 三种访问级别

ID	用例名称	前置条件	操作	期望结果	标签
T3.1.1	FREE 模式加密	明文数据	Encrypt(data, FREE)	D_node=1, 任何人可计算 aes_key, 输出格式: nonce(12B) ciphertext tag(16B)	[unit]
T3.1.2	PAID 模式加密	明文数据, D_node, P_node	Encrypt(data, PAID)	输出格式: nonce(12B) ciphertext tag(16B), 需 D_node 或 capsule 解密	[unit]

ID	用例名称	前置条件	操作	期望结果	标签
T3.1.3	PRIVATE 模式加密	明文数据, D_node, P_node	Encrypt(data, PRIVATE)	同 PAID 格式, 元数据同样加密 (enc_payload)	[unit]
T3.1.4	访问级别升级	PAID 密文	Decrypt → Re-encrypt(PRIVATE)	新密文可解密, 原密文仍有效	[integration]
T3.1.5	访问级别降级	PRIVATE 密文	Decrypt → Re-encrypt(FREE)	D_node=1, 任何人可解密	[integration]

T3.2: 密钥派生

ID	用例名称	前置条件	操作	期望结果	标签
T3.2.1	aes_key 派生确定性	D_node, P_node, key_hash	HKDF(ECDH(D_node, P_node).x, key_hash)	确定性输出 (相同输入→相同 aes_key)	[unit]
T3.2.2	FREE 模式密钥可公开计算	P_node (公开)	HKDF(P_node.x, key_hash)	与 D_node=1 计算结果一致	[unit]
T3.2.3	key_hash 双重哈希	明文数据	SHA256(SHA256(plaintext))	32 字节, 用于 KDF salt 和内容完整性验证	[unit]
T3.2.4	不同节点密钥隔离	不同 D_node	HKDF(ECDH(D1, P1).x, kh) vs HKDF(ECDH(D2, P2).x, kh)	两个 aes_key 不同	[property]
T3.2.5	BIP32 可推导性	父 S_node, child offset	$S_{child} = S_{parent} + offset \times P_{buyer}$	子密钥可从父密钥派生 (非硬化)	[property]

T3.3: AES-256-GCM 加解密

ID	用例名称	前置条件	操作	期望结果	标签
T3.3.1	正常加解密	任意明文			[unit]

ID	用例名称	前置条件	操作	期望结果	标签
			encrypt → decrypt	解密后与原文一致	
T3.3.2	空数据加密	空字节切片	encrypt([]byte{})	成功, 输出仅含 nonce+tag (28B)	[edge]
T3.3.3	大文件加密	10MB 随机数据	encrypt → decrypt	正确还原, 无截断	[edge]
T3.3.4	nonce 唯一性	同一密钥加密两次	encrypt(data) × 2	两次 nonce 不同, 密文不同	[property]
T3.3.5	tag 篡改检测	有效密文	修改最后 16B → decrypt	返回认证失败错误	[security]
T3.3.6	密文截断检测	有效密文	截断最后 1B → decrypt	返回解密失败错误	[security]
T3.3.7	nonce 篡改检测	有效密文	修改前 12B → decrypt	返回解密失败错误	[security]

T3.4: 错误路径 (新增)

ID	用例名称	前置条件	操作	期望结果	标签
T3.4.1	错误密钥解密	密钥 A 加密	密钥 B 解密	返回认证失败, 无部分明文泄漏	[security]
T3.4.2	nil 密钥	无密钥	encrypt(data, nil key)	返回 invalid key 错误	[edge]
T3.4.3	密文过短	长度 < 28B	decrypt(shortData)	返回 ciphertext too short 错误	[edge]

T4. Vault 管理

设计参照: 第二节 | 代码文件: `src/internal/method42/vault_test.go` | 现有/目标: 24 / 32

子类别	说明	现有	新增
T4.1 创建/列表/切换	vault create/list/switch 完整流程	8	+1
T4.2 持久化	钱包文件写入/读取一致性	6	+1
T4.3 密钥隔离	不同 vault 的密钥树完全独立	5	+1
T4.4 费用链	account 0 作为费用链的特殊行为	3	+1
T4.5 名称校验	非法名称 (空/特殊字符/重复) 拒绝	2	+1
T4.6 Seed 加密 (Argon2id)	Argon2id 参数验证、密码隔离、错误密码拒绝	0	+3

T4.6: Seed 加密 (Argon2id) (新增)

ID	用例名称	前置条件	操作	期望结果	标签
T4.6.1	Argon2id 参数验证	新建 vault	vault create → 检查 wallet.enc	Argon2id 参数: time=3, memory=65536 (64MB), parallelism=4; salt 随机 16 字节	[unit]
T4.6.2	不同密码产生不同密文	两组不同密码	vault create × 2 → 比较 wallet.enc	salt 不同, derived_key 不同, 密文不同	[property]
T4.6.3	错误密码解密失败	已创建 vault	Argon2id(wrong_password, salt) → 解密 wallet.enc	解密失败, 返回认证错误, 无部分明文泄漏	[security]

T5. 握手协议

设计参照: 第十三-B.B节 | 代码文件: `src/internal/method42/handshake_test.go` | 现有/目标: 8 / 17

子类别	说明	现有	新增
T5.1 完整握手	三阶段 ECDH 双向认证完整流程	2	+1
T5.2 session_key 派生	SHA256(ECDH_shared_x nonce_b nonce_s)	2	+2
T5.3 HMAC 验证	正确/错误 HMAC, 重放 nonce	2	+2
T5.4 会话建立	session token 生成与过期	1	+2
T5.5 错误路径	无效公钥, 阶段顺序错误, 超时	1	+2

T6. Key Capsule & HTLC

设计参照: 第十三-B.D节 | 代码文件: `src/internal/method42/keycapsule_test.go` | 现有/目标: 9 / 18

子类别	说明	现有	新增
T6.1 Capsule 生成	seller_mask XOR file_key = capsule	3	+1
T6.2 Capsule 验证	SHA256(capsule) = capsule_hash	2	+1
T6.3 密钥恢复	buyer_mask = ECDH(D_buyer, P_node); file_key =	2	+2

子类别	说明	现有	新增
	capsule XOR buyer_mask		
T6.4 HTLC 脚本	OP_SHA256 preimage check + 双花路径	1	+1
T6.5 超时路径	144 区块后 buyer 可 回收资金	1	+2
T6.6 htlc_tx 必填验证	htlc_tx 为必填字段, Seller 必须验证链上 存在	0	+2

T6.6: htlc_tx 必填验证 (新增)

ID	用例名称	前置条件	操作	期望结果	标签
T6.6.1	htlc_tx 缺失拒绝	已握手, 有效 txid	POST / buy/{txid} 不包含 htlc_tx 字 段	返回 htlc_tx required 错误, 拒 绝返回 capsule	[security]
T6.6.2	Seller 验证 htlc_tx 链上存在	已握手, 有效 txid + htlc_tx	POST / buy/{txid} 包含 htlc_tx	Seller 查询链上确 认 htlc_tx 存在后 才返回 capsule	[security]

T7. Key Cache

设计参照: 第十一节 | 代码文件: `src/internal/method42/keycache_test.go` | 现有/目标: 8 / 12

子类别	说明	现有	新增
T7.1 缓存读写	Put/Get 完整流程	3	+1

子类别	说明	现有	新增
T7.2 缓存路径	~/.bitfs/cache/keys/ 目录结构	2	+1
T7.3 缓存命中/未命中	已缓存密钥直接返回, 未缓存返回 nil	2	+1
T7.4 持久化	进程重启后缓存仍有效	1	+1

T8. Metanet 节点与构建器

设计参照: 第四-B节 | 代码文件: `src/internal/metanet/metanet_test.go` | 现有/目标: 34 / 49

子类别	说明	现有	新增
T8.1 BuildCreateRoot	根目录创建交易的完整结构	6	+2
T8.2 BuildCreateChild	FILE/DIR/LINK 子节点创建, OP_RETURN 格式	8	+3
T8.3 BuildSelfUpdate	父目录更新 (children 列表, next_child_index)	6	+2
T8.4 OP_RETURN 解析	Metanet 前缀 + Protobuf payload 解析	5	+3
T8.5 UTXO 管理	Output 2 刷新 P_parent, 自持续链	4	+2
T8.6 交易签名	P2PKH 签名验证, Input/Output 数量正确	5	+3

T9. 文件系统操作 (14 种) ★

设计参照：第四-B节，第三节 代码文件： `src/internal/metanet/fs_test.go` 测试函数数：现有 46 / 目标 73

T9.1: put (新建文件)

ID	用例名称	前置条件	操作	期望结果	标签
T9.1.1	标准 put	已有 DIR 父节点	put(“test.txt” , data)	生成 2 笔交易 (CreateChild + SelfUpdate), parent.children 新增 ChildEntry	[unit]
T9.1.2	重复文件名	同名文件已存在	put(“dup.txt” , data)	返回 file already exists 错误	[edge]
T9.1.3	加密 put	access=PAID	put + encrypt	密文格式 nonce ct tag, key_hash 正确	[unit]
T9.1.4	空文件	data=[]byte{}	put(“empty.txt” , nil)	成功, file_size=0	[edge]
T9.1.5	特殊文件名	含中文/空格/ unicode	put(“测试文 件.txt” , data)	成功, ChildEntry.name 保留原始字符	[edge]

T9.2: mkdir (新建目录)

ID	用例名称	前置条件	操作	期望结果	标签
T9.2.1	标准 mkdir	已有 DIR 父节点	mkdir(“subdir”)	生成 CreateChild(DIR) + SelfUpdate	[unit]
T9.2.2	嵌套创建	深度 5 层	mkdir -p a/b/c/ d/e	逐层创建, 每层 2 笔交易	[edge]

ID	用例名称	前置条件	操作	期望结果	标签
T9.2.3	重复目录名	同名目录已存在	mkdir(“dup”)	返回 directory already exists 错误	[edge]

T9.3: rm (删除文件)

ID	用例名称	前置条件	操作	期望结果	标签
T9.3.1	标准 rm	文件存在于 parent	rm(“test.txt”)	parent.children 移除对应 ChildEntry, 生成 SelfUpdate	[unit]
T9.3.2	rm 不存在文件	文件名不在 parent	rm(“ghost.txt”)	返回 file not found 错误	[edge]
T9.3.3	rm 硬链接目标	文件有 2 个硬链接	rm 其中一个	另一个硬链接仍有效, P_node 未删	[unit]

T9.4: rmdir (删除目录)

ID	用例名称	前置条件	操作	期望结果	标签
T9.4.1	空目录删除	DIR 无子节点	rmdir(“empty”)	成功移除	[unit]
T9.4.2	非空目录拒绝	DIR 含子文件	rmdir(“notempty”)	返回 directory not empty 错误	[edge]

T9.5: mv (移动/重命名)

ID	用例名称	前置条件	操作	期望结果	标签
T9.5.1	重命名	同目录下	mv(“a.txt” , “b.txt”)	ChildEntry.name 更新, P_node 不变	[unit]
T9.5.2	跨目录移动				[unit]

ID	用例名称	前置条件	操作	期望结果	标签
		源/目标目录都存在	mv(“/dir1/a.txt” , “/dir2/a.txt”)	目标新建节点 (新 P_node), 源节点变 SOFT 链接指向新节点, 保持 HD 树镜像	
T9.5.3	目标已存在	目标同名文件存在	mv(“a.txt” , “exist.txt”)	返回 target already exists 错误	[edge]
T9.5.4	mv 保留元数据	文件带 keywords, description	mv → 检查目标	所有元数据保持不变	[unit]

T9.6: cp (复制)

ID	用例名称	前置条件	操作	期望结果	标签
T9.6.1	标准 cp	源文件存在	cp(“src.txt” , “dst.txt”)	新 P_node, 新 ChildEntry, 解密 → 重加密完整流程	[unit]
T9.6.2	cp 加密文件	PAID 源	cp → 检查目标	目标独立密钥, 可独立解密	[unit]
T9.6.3	cp 目录递归	目录含多层子节点	cp -r dir1 dir2	递归复制, 每个节点新 P_node	[edge]

T9.7: link_hard (硬链接)

ID	用例名称	前置条件	操作	期望结果	标签
T9.7.1	创建硬链接	源文件存在	link(“src.txt” , “hard.txt” , HARD)	两个 ChildEntry → 同一 P_node	[unit]
T9.7.2	删除原始文件	有硬链接	rm 原始 → 通过链接访问	链接仍可正常读取	[unit]

T9.8: link_soft (软链接)

ID	用例名称	前置条件	操作	期望结果	标签
T9.8.1	创建软链接	目标路径存在	link(“target” , “sym” , SOFT)	ChildEntry 存储目标 P_node	[unit]
T9.8.2	悬空链接	删除目标后	通过软链接访问	返回 target not found 错误	[edge]
T9.8.3	链式解析	A→B→C	解析 A	最终到达 C, 无循环	[edge]

T9.9: link_soft_remote (远程软链接)

ID	用例名称	前置条件	操作	期望结果	标签
T9.9.1	远程链接创建	domain + 路径	link(“example.com/file” , “remote” , SOFT_REMOTE)	ChildEntry 含 remote_domain + remote_path	[unit]

T9.10: encrypt (加密文件)

ID	用例名称	前置条件	操作	期望结果	标签
T9.10.1	FREE→PAID	FREE 文件	encrypt(“file.txt”)	access 变更为 PAID, 重新加密 (新 D_node ECDH), key_hash 不变	[unit]
T9.10.2	已加密文件	PAID 文件	encrypt(“file.txt”)	返回 already encrypted 错误	[edge]

T9.11: decrypt (解密文件)

ID	用例名称	前置条件	操作	期望结果	标签
T9.11.1	PAID→FREE	加密文件	decrypt(“file.txt”)	access=FREE, D_node=1 重新加密 (任何人可解密)	[unit]
T9.11.2	解密 FREE 文件	FREE 文件	decrypt(“file.txt”)	返回 not encrypted 错误	[edge]

T9.12: sell (设置价格)

ID	用例名称	前置条件	操作	期望结果	标签
T9.12.1	文件定价	PAID 文件	sell(“file.txt” , 100)	price_per_kb=100 sat/KB	[unit]
T9.12.2	目录定价继承	目录含子文件	sell(“/dir” , 50)	子文件继承 price_per_kb=50	[unit]
T9.12.3	零价格	设置 price=0	sell(“file.txt” , 0)	返回 invalid price 错误	[edge]

T9.13: update_self (自更新)

ID	用例名称	前置条件	操作	期望结果	标签
T9.13.1	更新元数据	已有节点	update(keywords=[“new”])	SelfUpdate 交易, keywords 更新	[unit]

T9.14: version (版本管理)

ID	用例名称	前置条件	操作	期望结果	标签
T9.14.1	版本链查询		listVersions(P_node)		[unit]

ID	用例名称	前置条件	操作	期望结果	标签
		同一 P_node 多次更新		按 block height/TTOR 排序	
T9.14.2	最新版本	3 个版本	resolve(P_node)	返回 block height 最高的版本	[unit]

T10. Metanet 解析器

设计参照: 第四-B节 | 代码文件: `src/internal/metanet/metanet_test.go` | 现有/目标: 10 / 19

子类别	说明	现有	新增
T10.1 路径解析	“/dir/subdir/file.txt” → 逐级 P_node 解析	3	+2
T10.2 软链接跟随	遇 SOFT link 自动解析目标 P_node	2	+2
T10.3 版本解析	同一 P_node 的多版本中选择最新	2	+2
T10.4 深层路径	5+ 层嵌套路径解析	1	+1
T10.5 错误路径	不存在路径, 中间节点非 DIR, 循环链接	2	+2

T11. 内容寻址存储

设计参照: 第七节 | 代码文件: `src/internal/storage/store_test.go` | 现有/目标: 22 / 26

子类别	说明	现有	新增
T11.1 Store/Retrieve	SHA-256 键存储与读取	6	+1
T11.2 去重	相同内容只存一次, 引用计数	5	+1
T11.3 完整性	读取时校验 hash 是否匹配	4	+1
T11.4 原子写入	写入中断不留半成品文件	4	+0
T11.5 并发安全	多 goroutine 同时读写	3	+1

T12. SPV 客户端

设计参照: 第七节 | 代码文件: `src/internal/spv/spv_test.go` | 现有/目标: 41 / 50

子类别	说明	现有	新增
T12.1 Merkle proof 验证	给定 tx + proof + root → 验证通过/失败	10	+2
T12.2 Header chain	区块头链验证 (prev_hash, difficulty)	8	+2
T12.3 Tx 存储	交易本地存储与检索	8	+1
T12.4 proof 篡改	修改 proof 中间节点 → 验证失败	6	+2
T12.5 链恢复	从本地 tx + proof 重建节点状态	5	+1
T12.6 边界条件	空 proof, 单节点树, 极深树	4	+1

T13. DNSLink

设计参照: 第六节 | 代码文件: `src/internal/dnslink/dnslink_test.go` | 现有/目标: 4 / 13

子类别	说明	现有	新增
T13.1 TXT 记录解析	<code>_bitfs_pubkey</code> TXT → P_node 公钥	1	+2
T13.2 SRV 记录	<code>_bitfs._tcp</code> SRV → 端点列表 (priority/weight/port)	1	+2
T13.3 双向验证	DNS 记录中公钥与链上 P_node 匹配	1	+2

子类别	说明	现有	新增
T13.4 多端点	多个 SRV 记录的优先级排序, CDN 场景	1	+1
T13.5 错误处理	DNS 查询超时, 无记录, 格式错误	0	+2

T14. URI 寻址

设计参照: 第六节 | 代码文件: `src/internal/addressing/uri_test.go` | 现有/目标: 11 / 14

子类别	说明	现有	新增
T14.1 bitfs:// 解析	scheme/host/path/query 各组件	5	+1
T14.2 简写格式	txid/path, pnode/path, domain/path	4	+1
T14.3 错误格式	非法 scheme, 缺失 host, 路径注入	2	+1

T15. Daemon HTTP ★

设计参照: 第十三-B.A节 代码文件: `src/internal/daemon/daemon_test.go` 测试函数数: 现有 99 / 目标 111

T15.1: 健康检查

ID	用例名称	前置条件	操作	期望结果	标签
T15.1.1	GET /health	daemon 正常运行	GET /health	200, JSON 含 vault 信息	[unit]

T15.2: 数据服务

ID	用例名称	前置条件	操作	期望结果	标签
T15.2.1	正常获取	有效 hash, 配额内	GET / data/ {hash}	200, 返回加密内容	[unit]
T15.2.2	配额超限触发 402	日配额已用完	GET / data/ {hash}	402, X-Payment- Required header, JSON invoice	[unit]
T15.2.3	不存在的 hash	无效 hash	GET / data/ {invalid}	404, content not found	[edge]

T15.3: 元数据查询

ID	用例名称	前置条件	操作	期望结果	标签
T15.3.1	正常查询	有效 pnode + 路径	GET / meta/ {pnode}/ {path}	200, JSON 含节点 元数据	[unit]
T15.3.2	不存在路径	无效路径	GET / meta/ {pnode}/ ghost	404	[edge]

T15.4: x402 支付

ID	用例名称	前置条件	操作	期望结果	标签
T15.4.1	正常支付	有效 invoice_id	POST /pay/ {invoice_id}	200, 配额恢复	[unit]
T15.4.2	已支付 invoice	重复支付	POST /pay/ {invoice_id}	返回 already paid 错误	[edge]

ID	用例名称	前置条件	操作	期望结果	标签
T15.4.3	过期 invoice	>300 秒	POST /pay/{invoice_id}	返回 invoice expired 错误	[edge]

T15.5: 握手端点

ID	用例名称	前置条件	操作	期望结果	标签
T15.5.1	正常握手	有效公钥 + nonce	POST / handshake	200, 返回 server nonce + HMAC	[unit]
T15.5.2	无效 HMAC	篡改 HMAC	POST / handshake (phase 3)	返回 authentication failed 错误	[security]

T15.6: 购买流程

ID	用例名称	前置条件	操作	期望结果	标签
T15.6.1	获取价格	已握手, 有效 txid	GET /buy/{txid}	200, JSON 含 price_per_kb, capsule_hash, total_price	[unit]
T15.6.2	提交 HTLC	HTLC 已广播	POST / buy/{txid} + htlc_tx (必填)	200, Seller 验证 htlc_tx 链上存在后 返回 capsule (preimage)	[unit]
T15.6.3	未握手即购买	无有效 session	GET /buy/{txid}	返回 session required 错误	[edge]
T15.6.4	HTLC 未确认	HTLC 未上链	POST / buy/{txid} + htlc_tx	返回 HTLC not confirmed 错误	[edge]
T15.6.5	缺少 htlc_tx	已握手, 有效 session	POST / buy/{txid}	返回 htlc_tx required 错误, 拒绝返回 capsule	[security]

ID	用例名称	前置条件	操作	期望结果	标签
			不包含 htlc_tx		

T15.7: Content Negotiation (新增)

ID	用例名称	前置条件	操作	期望结果	标签
T15.7.1	HTML 响应	Accept: text/html	GET /path	HTML 含 4 个 WebMCP tool 声明	[unit]
T15.7.2	Markdown 响应	Accept: text/markdown	GET /path	Markdown 含 CLI 命令示例	[unit]
T15.7.3	JSON 响应	Accept: application/json	GET /path	结构化 JSON 数据	[unit]

T16. CLI 工具

设计参照: 第八节, 第九-B节 | 代码文件: `src/cmd/*_test.go` | 现有/目标: 139 / 181

子类别	说明	现有	新增
T16.1 bls	列目录: 格式化输出, -json, -long, -a 隐藏文件	9	+4
T16.2 bcat	输出内容: stdout 输出, 二进制检测, 管道兼容	9	+3
T16.3 bget	下载文件: 输出到文件, -buy 购买流程, 断点续传	12	+5
T16.4 bstat	元数据: 所有字段显示, -json, 链接信息	9	+4

子类别	说明	现有	新增
T16.5 btree	目录树: 递归深度, -level, 排序	10	+3
T16.6 bitfs put	上传: 加密选项, mime 检测, 元数据标志	10	+4
T16.7 bitfs mkdir/rm/rmdir	目录管理: 标准行为 + 错误处理	8	+3
T16.8 bitfs mv/cp/link	文件操作: 三种操作 + 参数组合	10	+3
T16.9 bitfs encrypt/decrypt	加密管理: 模式切换 + 错误路径	8	+2
T16.10 bitfs sell	定价: 价格设置, 继承, sales 查看	6	+2
T16.11 bitfs vault/wallet	钱包管理: init, create, list, switch	12	+2
T16.12 bitfs daemon	启停: start, stop, 端口冲突	6	+1
T16.13 bget -buy	完整购买: 握手→定价→HTLC→capsule→解密	15	+4
T16.14 通用选项	-output json/table/plain, -help, -version	15	+2

T17. CLI 通用

设计参照: 第八节 | 代码文件: `src/internal/cli/common_test.go` | 现有/目标: 16 / 22

子类别	说明	现有	新增
T17.1 输出格式化	FormatTable, FormatJSON, FormatPlain	6	+2

子类别	说明	现有	新增
T17.2 路径解析	ResolvePath: 绝对/相对/~ 展开	4	+2
T17.3 错误显示	用户友好错误信息, exit code 映射	4	+1
T17.4 HTTP 客户端	daemon 连接, 超时, 重试	2	+1

T18. 配置与错误处理

设计参照: 第十六节 | 代码文件: `src/internal/config/config_test.go` | 现有/目标: 33 / 38

子类别	说明	现有	新增
T18.1 配置加载	~/.bitfs/config.yaml 读取, 默认值, 环境变量覆盖	10	+1
T18.2 Exit codes	0/1/2/3/4/5/6/7 全部覆盖 (见系统设计 Exit Codes)	8	+1
T18.3 重试策略	指数退避, 最大重试次数, 可重试错误判断	8	+1
T18.4 错误包装	fmt.Errorf + %w, errors.Is/errors.As 链	7	+2

T19. Shell 交互

设计参照: 第十节, 第九-B.C节 | 代码文件: `src/internal/shell/shell_test.go` | 现有/目标: 21 / 28

子类别	说明	现有	新增
T19.1 命令解析	30+ 命令的参数解析与路由	8	+2
T19.2 状态管理	cwd, 连接状态, prompt 更新	5	+2
T19.3 本地命令	lcd, lpwd, lls, ! (shell escape)	4	+1

子类别	说明	现有	新增
T19.4 批量操作	mget, mput 多文件传输	2	+1
T19.5 REPL 循环	EOF 退出, 空行忽略, 历史记录	2	+1

T20. 集成/端到端 ★

设计参照: 全部章节 代码文件: `src/internal/integration/e2e_test.go` 测试函数数: 现有 7 / 目标 27

T20.1: 钱包与文件系统 (现有 3, 新增 5)

ID	用例名称	前置条件	操作	期望结果	标签
T20.1.1	完整文件生命周期	钱包已初始化	init → vault → put → bls → bcat → verify	内容一致, 元数据 正确	[integration]
T20.1.2	多 Vault 隔离	两个 vault	vault A put → vault B put → 交 叉访问	vault 间不可见	[integration]
T20.1.3	深层目录操作	空 vault	mkdir a/ b/c → put file → mv → cp → verify	所有操作正确, 树 结构一致	[integration]
T20.1.4	版本链完整流程	已有文件	put v1 → update → put v2 → resolve	两个版本均可访问	[integration]

ID	用例名称	前置条件	操作	期望结果	标签
			latest → resolve v1		
T20.1.5	内容去重验证	空存储	put 相同内容两次 → 检查存储	存储中只有一个副本	[integration]

T20.2: 加密全流程 (现有 2, 新增 3)

ID	用例名称	前置条件	操作	期望结果	标签
T20.2.1	三种级别 round-trip	三种 access	encrypt → store → retrieve → decrypt	每种级别正确还原	[integration]
T20.2.2	加密文件复制	PAID 文件	cp → 用目标密钥解密 → verify	独立密钥, 内容一致	[integration]
T20.2.3	密钥恢复 (助记词)	钱包已初始化	删除钱包 → 用助记词恢复 → 解密旧文件	成功解密	[integration]

T20.3: 买卖全流程 (现有 1, 新增 5)

ID	用例名称	前置条件	操作	期望结果	标签
T20.3.1	完整购买流程	seller 有加密文件	sell → buyer handshake → GET / buy → HTLC →	buyer 获得明文	[integration]

ID	用例名称	前置条件	操作	期望结果	标签
			POST /buy → decrypt		
T20.3.2	密钥缓存复用	已购买过	再次 GET / data → 用 缓存密钥解 密	无需重复支付	[integration]
T20.3.3	HTLC 超时回收	buyer 未提交 HTLC	等待 144 块	buyer 资金回收	[integration]
T20.3.4	目录递归定价	目录设置 sell	购买子文件 → 检查价格	子文件继承父目录 价格	[integration]
T20.3.5	x402 配额流程	免费配额耗尽	GET /data → 402 → POST /pay → GET / data	支付后正常获取	[integration]

T20.4: Daemon 与网络 (现有 1, 新增 4)

ID	用例名称	前置条件	操作	期望结果	标签
T20.4.1	Daemon 启停	无	start → health → serve → stop	正常启停, 端口释 放	[integration]
T20.4.2	DNSLink 发布解析	DNS mock	publish → resolve via domain → access	端到端域名访问	[integration]
T20.4.3	Content Negotiation	daemon 运行	同一 URL 不同 Accept → 不同格式	HTML/ Markdown/JSON 三种	[integration]

ID	用例名称	前置条件	操作	期望结果	标签
T20.4.4	Shell 会话	daemon 运行	open → ls → cd → cat → put → close	REPL 完整流程	[integration]

T20.5: SPV 验证 (新增 3)

ID	用例名称	前置条件	操作	期望结果	标签
T20.5.1	交易验证链	多笔关联交易	创建交易 → 生成 proof → 验证 → 篡 改 → 检测	完整性保证	[integration]
T20.5.2	离线模式	有本地缓存	断开网络 → 读取缓 存数据 → verify	离线可用	[integration]
T20.5.3	恢复重建	本地 tx + proof	重建整棵 Metanet 树	与原始一致	[integration]

T21. 属性不变量 ★

设计参照: 多处 (交叉引用见下) 代码文件: (新建) `src/internal/integration/property_test.go` 测试函数数: 现有 0 / 目标 23

属性不变量测试验证系统的数学和密码学性质, 使用随机输入 (固定种子) 反复验证。

T21.1: 密码学不变量

ID	用例名称	属性	设计参照	标签
T21.1.1	ECDH 对称性	$D_a \times P_b = D_b \times P_a$ (任意密钥对)	五, 二-B.D	[property]
T21.1.2	加密可逆性	<code>decrypt(encrypt(data, key), key) = data</code> (任意 data)	五	[property]
T21.1.3	密钥派生确定性	<code>HKDF(ECDH(D_node, P_node).x, key_hash)</code> → 相同输出 (100 次)	二-B.D	[property]
T21.1.4	BIP39 恢复确定性	<code>mnemonic</code> → <code>seed</code> → <code>wallet</code> → 相同密钥树 (10 组)	二-B.A	[property]
T21.1.5	AES-GCM 认证性	任意篡改 → tag 验证失败 (fuzz 100 次)	五	[property]
T21.1.6	Capsule 正确性	<code>capsule XOR</code> <code>buyer_mask = file_key</code> (100 组)	十三-B.D	[property]
T21.1.7	Session key 确定性	<code>SHA256(shared_x </code> <code>nonce_b nonce_s)</code> 一致 (双方计算)	十三-B.B	[property]
T21.1.8	HTLC preimage 一致性	<code>SHA256(capsule) =</code> <code>capsule_hash</code> (所有交换)	十三-B.D	[property]

T21.2: 数据结构不变量

ID	用例名称	属性	设计参照	标签
T21.2.1	Protobuf round-trip	<code>marshal(unmarshal(bytes))</code> <code>= bytes</code>	四	[property]
T21.2.2	内容寻址唯一性		七	[property]

ID	用例名称	属性	设计参照	标签
		SHA256(SHA256(plaintext)) 唯一标识内容 (双哈希, 碰撞检测)		
T21.2.3	去重一致性	Store(data) × N → 存储中仅 1 副本	七	[property]
T21.2.4	URI round-trip	parse(format(uri)) = uri	六	[property]

T21.3: 文件系统不变量

ID	用例名称	属性	设计参照	标签
T21.3.1	Index 单调递增	next_child_index 只增不减	四-B	[property]
T21.3.2	父子一致性	parent.children 包含所有子节点 P_node	三, 四-B	[property]
T21.3.3	UTXO 链连续性	每个 Output 2 刷新到同一 P_node	四-B	[property]
T21.3.4	硬链接共享	硬链接的多个 ChildEntry → 同一 P_node	三	[property]
T21.3.5	版本排序一致	block height 排序 = TTOR 排序 (Last-Write-Wins)	十二	[property]
T21.3.6	Vault 隔离性	不同 vault → 不同密钥树, 不同 P_root	二	[property]
T21.3.7	rm 原子性	rm 后 parent.children 不含已删条目	三	[property]

T21.4: 网络不变量

ID	用例名称	属性	设计参照	标签
T21.4.1	x402 配额日重置	free_quota_kb 每日 每 IP 重置	十三-B.C	[property]
T21.4.2	DNS 双向验证	published pubkey = 链上 P_node 公钥	六	[property]
T21.4.3	FREE 公开可解密	FREE 模式 D_node=1, aes_key = KDF(P_node, key_hash) 可公开计 算, 仍经 AES-GCM 加密 (非 identity)	五	[property]
T21.4.4	aes_key 派生确定性	aes_key = KDF(ECDH(D_node, P_node), key_hash) — D_node 或 key_hash 任一变→ 输出变	二-B.D	[property]

T22. 安全性 ★

设计参照: 第五节, 第十一节, 第十三-B节 代码文件: (新建) `src/internal/integration/security_test.go` 测试函数数: 现有 0 / 目标 17

安全性测试验证系统在对抗性条件下的行为, 确保攻击者无法获取未授权数据。

T22.1: 密码学安全

ID	用例名称	前置条件	操作	期望结果	标签
T22.1.1	错误密钥解密	密钥 A 加密	密钥 B 解密	认证失败, 无部分明文泄漏	[security]

ID	用例名称	前置条件	操作	期望结果	标签
T22.1.2	密文篡改检测	有效密文	随机翻转 1 bit	AES-GCM tag 校验失败	[security]
T22.1.3	nonce 重放	截获 nonce	重放相同 nonce	系统检测到重复 nonce 或密文不同	[security]
T22.1.4	密钥材料清理	加密操作完成	检查内存	密钥材料在使用后被清零	[security]

T22.2: 协议安全

ID	用例名称	前置条件	操作	期望结果	标签
T22.2.1	MITM 握手攻击	正常握手进行中	修改 HMAC 值	双方检测到篡改, 握手终止	[security]
T22.2.2	过期 session	session 超时后	使用旧 token 访问	返回 session expired, 拒绝服务	[security]
T22.2.3	Capsule 篡改	seller 返回 capsule	修改 capsule → SHA256 验证	hash 不匹配, buyer 拒绝接受	[security]
T22.2.4	HTLC 超时保护	buyer 广播 HTLC	seller 不揭示 preimage, 等 144 块	buyer 通过超时路径回收资金	[security]
T22.2.5	费用限制	正常交易	检查交易费	费用在预期范围内, 无费用窃取	[security]

T22.3: 输入验证

ID	用例名称	前置条件	操作	期望结果	标签
T22.3.1	路径遍历	文件名 “.././../etc/passwd”	put/ resolve	拒绝, 返回 invalid path 错误	[security]

ID	用例名称	前置条件	操作	期望结果	标签
T22.3.2	超大 payload	构造 >100MB protobuf	提交交易	拒绝, 返回 payload too large 错误	[security]
T22.3.3	恶意元数据	metadata 含注入字符	存储 → 检索 → 显示	元数据被安全处理, 无 injection	[security]
T22.3.4	CLI 参数注入	参数含 shell 特殊字符	执行命令	参数被正确引用, 无命令注入	[security]

T22.4: 访问控制

ID	用例名称	前置条件	操作	期望结果	标签
T22.4.1	未认证端点访问	无 session token	访问受保护的 /buy 端点	401, 拒绝服务	[security]
T22.4.2	跨 Vault 访问	Vault A 的密钥	解密 Vault B 的文件	失败, 密钥不匹配	[security]
T22.4.3	并发加密隔离	两个 goroutine	同时加密不同文件	各自独立, 无状态泄漏	[security]
T22.4.4	PRIVATE 元数据保护	PRIVATE 文件	无密钥查询元数据	元数据不可见	[security]

T23. 会话管理 (Lock/Unlock) ★

设计参照: 第二十一节 代码文件: `src/internal/method42/session_test.go` 测试函数数: 现有 0 / 目标 24

T23.1: unlock 命令

ID	用例名称	前置条件	操作	期望结果	标签
T23.1.1	基本 unlock				[unit]

ID	用例名称	前置条件	操作	期望结果	标签
		vault 已初始化, 未 unlock	unlock (无 duration)	session 文件创建, expires_at=0, 权限 0600	
T23.1.2	带时长 unlock	vault 已初始化	unlock – duration 30m	session 文件创建, expires_at = created_at + 1800	[unit]
T23.1.3	密码错误	vault 已初始化	unlock, 输入错误密码	返回 decryption failed 错误, 不创建 session 文件	[unit]
T23.1.4	重复 unlock	已 unlock 状态	再次 unlock	session 文件被覆盖更新 (新 created_at)	[edge]
T23.1.5	unlock + daemon 在线	daemon 运行中	unlock	daemon 收到 unlock RPC + session 文件同时创建	[integration]

T23.2: lock 命令

ID	用例名称	前置条件	操作	期望结果	标签
T23.2.1	基本 lock	session 文件存在	lock	session 文件被安全覆写后删除	[unit]
T23.2.2	lock + daemon 在线	daemon 运行中且已 unlock, session 文件存在	lock	daemon master key 清零 + session 文件删除 (两个独立动作)	[integration]
T23.2.3	未 unlock 时 lock	无 session 文件, daemon 未运行	lock	无操作, 无错误 (幂等)	[edge]

T23.3: Session 文件管理

ID	用例名称	前置条件	操作	期望结果	标签
T23.3.1	文件权限	unlock 创建 session 文件	检查文件权限	权限为 0600 (owner 读写)	[security]
T23.3.2	过期检测	session 文件 expires_at 已过期	CLI 工具读取 session	检测到过期 → 安全覆写删除 → session 无效	[unit]
T23.3.3	永不过期	session 文件 expires_at=0	CLI 工具读取 session (任意时间后)	session 始终有效	[unit]
T23.3.4	文件损坏	session 文件内容非法 JSON	CLI 工具读取 session	视为无效, 安全删除损坏文件	[edge]
T23.3.5	encryption_key 正确性	session 文件存在	读取 encryption_key → Wallet.Load()	成功解密 seed, 与原始密码等效	[unit]

T23.4: CLI 密钥获取优先级

ID	用例名称	前置条件	操作	期望结果	标签
T23.4.1	优先级: daemon > session > prompt	daemon 在线已 unlock + session 文件存在	CLI 需要私钥	使用 daemon (不读 session 文件)	[integration]
T23.4.2	daemon 不可用 fallback	daemon 未运行, session 文件存在	CLI 需要私钥	从 session 文件读取 encryption key	[unit]
T23.4.3	全不可用 fallback	daemon 未运行, 无 session 文件	CLI 需要私钥	提示用户输入密码 (单次使用)	[unit]

T23.5: 安全覆写

ID	用例名称	前置条件	操作	期望结果	标签
T23.5.1	3 次零填充	session 文件存在 (含敏感数据)	secureDelete(path)	文件被写入 0x00 三次, 每次 fsync, 最后删除	[security]
T23.5.2	文件不存在	session 文件不存在	secureDelete(path)	无操作, 无错误 (幂等)	[edge]

T23.6: Argon2id 与会话安全 (新增)

ID	用例名称	前置条件	操作	期望结果	标签
T23.6.1	bitfs unlock 正确解锁 (Argon2id)	vault 已初始化	unlock, 输入正确密码	Argon2id(password, salt, time=3, mem=65536, p=4) 派生 derived_key, 成功解密 wallet.enc	[unit]
T23.6.2	bitfs lock 清除内存密钥	session 活跃	lock	lock 后任何需要密钥的操作返回 “wallet locked” 错误, 内存中无残留密钥	[security]
T23.6.3	Session 30 分钟自动过期	unlock -duration 30m	等待超过 30 分钟	超时后操作返回 “session expired, please unlock” 错误	[unit]
T23.6.4	Session 不存储密码原文	unlock 后	检查 session 文件内容	session 文件中不含 password 明文或 base64 编码的密码字段, 仅含 encryption_key	[security]
T23.6.5	错误密码解锁失败 (Argon2id)	vault 已初始化	unlock, 输入错误密码	Argon2id 派生的 key 无法解密 wallet.enc, 返回认证错误	[security]
T23.6.6		已有活跃 session			[edge]

ID	用例名称	前置条件	操作	期望结果	标签
	并发 session (同一 vault)		第二次 unlock (同一 vault)	第二次 unlock 替换/刷新现有 session, 旧 session 失效	

T24. 权限管理 (ACL + 群签名) ★

设计参照: 第二十二节 代码文件: `src/internal/acl/acl_test.go` 测试函数数: 现有 0 / 目标 10

ACL 权限管理测试验证群签名/群加密与 POSIX ACL 风格权限控制的正确性。

T24.1: ACL 字段与存储

ID	用例名称	前置条件	操作	期望结果	标签
T24.1.1	acl_ref 字段写入 Protobuf	已有 Metanet 节点	设置 acl_ref → 序列化 BitFSPayload	BitFSPayload 正确包含 acl_ref 字节, 反序列化后一致	[unit]
T24.1.2	空 acl_ref 表示无 ACL 限制	无 ACL 的文件	检查 acl_ref 字段	默认行为: 按 access 模式 (FREE/PAID/PRIVATE) 控制, 向后兼容	[unit]
T24.1.3	ACL 更新交易广播	已有 ACL 节点	Owner 更新 ACL 规则 → 广播	新 ACL 规则正确上链 (新 txid), 所有引用者通过 pubkey 自动解析到最新版本	[integration]

T24.2: BLS12-381 群签名

ID	用例名称	前置条件	操作	期望结果	标签
T24.2.1		HD seed 已生成		生成有效 BLS 密钥对, 与	[unit]

ID	用例名称	前置条件	操作	期望结果	标签
	BLS12-381 群签名密钥从 HD seed 派生		HKDF(seed, info= "bitfs-bls12-381")	secp256k1 密钥独立	
T24.2.2	群签名验证: 有效签名	群成员持有 credential	GroupSign(credential, message) → VerifyGroupSig(GPK, signature)	VerifyGroupSig 返回 true	[unit]
T24.2.3	群签名验证: 无效签名	非群成员或篡改签名	VerifyGroupSig(GPK, invalid_signature)	VerifyGroupSig 返回 false	[security]
T24.2.4	群签名匿名性: 验证者无法识别签名者身份	多个群成员	各成员分别签名 → 验证	所有签名均通过验证但验证者无法区分签名者身份	[property]

T24.3: 群加密

ID	用例名称	前置条件	操作	期望结果	标签
T24.3.1	群成员使用群公钥加密文件	群已创建, 含多个成员	GroupEncrypt(GPK, file_key) → 各成员 GroupDecrypt	所有群成员可用各自 credential 解密, 密文一致	[unit]
T24.3.2	非群成员尝试解密群加密文件	非群成员	GroupDecrypt(non_member_credential, ciphertext)	解密失败, 返回权限错误	[security]
T24.3.3	撤销群成员后	群成员 B 被撤销	撤销 B → GroupEncrypt 新文件 → B 尝试解密	被撤销成员 B 无法解密新	[security]

ID	用例名称	前置条件	操作	期望结果	标签
	加密新文件			文件, 其他成员正常解密	

T25. 收益权/ISO ★

设计参照: 第十一节 代码文件: `src/internal/revenue/iso_test.go` 测试函数数: 现有 0 / 目标 8

收益权表与 ISO (Initial Share Offering) 测试验证收益分配、股份管理和 Covenant 脚本正确性。

T25.1: Protobuf 字段

ID	用例名称	前置条件	操作	期望结果	标签
T25.1.1	revenue_share 字段写入 Protobuf	已有 Metanet 节点	设置 revenue_share=5000 → 序列化	值 5000 表示 50.00% 分成比例, 反序列化后一致	[unit]
T25.1.2	无 revenue_share 的文件不触发分成	revenue_share=0	购买该文件	全额归 Owner, 无股东分配逻辑	[unit]

T25.2: Share/Registry UTXO

ID	用例名称	前置条件	操作	期望结果	标签
T25.2.1	Share UTXO 正确编码持有人和份额	ISO 已创建	检查 Share UTXO Script	Script 包含 holder_pkh 和 share_amount, 输出 ≥ 546 sat	[unit]
T25.2.2					[unit]

ID	用例名称	前置条件	操作	期望结果	标签
	Registry UTXO 记录所有股东	ISO 已创建, 有多个股东	查询 Registry UTXO	股东列表完整, 份额总和 = total_shares (10000)	

T25.3: 收益分配

ID	用例名称	前置条件	操作	期望结果	标签
T25.3.1	收益分配: 金额 = 总收入 * 份额比例	股东持 50% 份额	10000 sat 收入触发分配	该股东收到 5000 sat, Covenant 验证通过	[unit]
T25.3.2	收益分配: 每个股东输出 >= 546 sat	多个小股东	小额收入触发分配	低于 546 sat dust limit 时累积到下次分配, 不生成无效输出	[edge]
T25.3.3	股份转让: Covenant 验证份额守恒	股东 A 持有 3000 份	A 转让 1000 份给 D	转让前后总份额不变 (10000), Registry 正确更新	[property]
T25.3.4	购买 PAID 文件触发自动分成	文件设有 revenue_share	Buyer 购买文件	Seller 收到扣除分成后的金额, 各股东按份额收到对应金额	[integration]

T26. 同步与批量发布 (bsync/bput) ★

设计参照: 第二十三节 代码文件: `src/internal/sync/bsync_test.go` 测试函数数: 现有 0 / 目标 6

bsync/bput 同步与上传工具测试, 验证三阶段流水线 (Scan→Diff→Apply) 与文件级同步的正确性。

T26.1: bsync 同步

ID	用例名称	前置条件	操作	期望结果	标签
T26.1.1	bsync 检测本地与链上差异	本地与远程存在差异	bsync ./project/bitfs://example.com/project/	正确列出新增、修改、删除的文件 (三方比较: local vs remote vs last_sync)	[unit]
T26.1.2	bsync -dry-run	本地与远程存在差异	bsync -dry-run ./project/bitfs://example.com/project/	仅显示差异摘要, 不执行任何链上操作, 不修改 sync state	[unit]

T26.2: bput 上传

ID	用例名称	前置条件	操作	期望结果	标签
T26.2.1	bput 将本地文件发布到链上	本地文件存在	bput ./readme.md bitfs://example.com/docs/readme.md	文件 Metanet 节点正确创建 (CreateChild + SelfUpdate), 内容可通过 bget 获取	[unit]
T26.2.2	bput -onchain 使用数据交易模式	本地文件存在	bput -onchain ./file.txt bitfs://example.com/file.txt	内容写入 OP_RETURN (链上存储), 非 daemon 本地存储	[unit]
T26.2.3	bput 大文件自动分片	超过单笔交易阈值的文件	bput ./large.bin bitfs://example.com/large.bin	自动创建多个 chunk 交易 + 元数据交易, 各 chunk 独立加密	[edge]
T26.2.4	bput 目录递归发布	含子目录的本地目录	bput -r ./src/bitfs://		[unit]

ID	用例名称	前置条件	操作	期望结果	标签
			example.com/ project/src/	子目录和文件按 DFS 顺序逐一创建 Metanet 节点	

T27. Paymail 集成 ★

设计参照: 第六节, 第十六-B节 代码文件: `src/internal/paymail/paymail_test.go` 测试函数数:
现有 0 / 目标 6

Paymail (bsvalias) 协议集成测试, 验证身份发现、公钥查询与 BitFS 协议的桥接。

T27.1: Capability 与 PKI

ID	用例名称	前置条件	操作	期望结果	标签
T27.1.1	.well-known/ bsvalias 返回 capabilities	daemon 运行, Paymail 已配 置	GET https:// example.com/.well- known/bsvalias	JSON 包含 bsvalias= "1.0" 和 pki, public- profile, verify 端 点模板	[unit]
T27.1.2	PKI endpoint 返回 vault P_root 公钥	vault 已初始化, Paymail 已绑 定	GET /api/v1/pki/ {alias}@domain	返回压缩公钥 (33 bytes hex), 与 vault P_root 公钥 完全匹配	[unit]

T27.2: 地址解析

ID	用例名称	前置条件	操作	期望结果	标签
T27.2.1	Paymail 地址 解析为 bitfs:// URI	DNS SRV + Paymail 服务 正常	解析 bitfs:// alice@example.com/ docs/readme.md	正确通过 SRV → capabilities → PKI → P_root, 最终解 析为对应 Metanet 路径	[integration]

ID	用例名称	前置条件	操作	期望结果	标签
T27.2.2	买方 Paymail 地址转换为公钥	买方有 Paymail 地址	通过 PKI 查询 bob@handcash.io	获取 P_buyer 压缩公钥, 可用于 HTLC 构建和 Method 42 握手	[unit]

T27.3: DNS 与派生

ID	用例名称	前置条件	操作	期望结果	标签
T27.3.1	DNS SRV 记录与 DNSLink 共存	同一域名配置 _bsvalias._tcp 和 _bitfs._tcp	分别解析 Paymail 和 DNSLink	两套 DNS 记录独立工作, Paymail 多用户 + DNSLink 单用户, 互不干扰	[integration]
T27.3.2	Paymail HD 派生 payout 地址	股东注册为 Paymail 地址	多次分红 → 每次解析 Paymail address	每次支付使用不同 HD 派生地址, 增强隐私; 无需更新链上 Registry	[unit]

Metanet Chain 测试用例已移至独立文档: [../metanet/4-TestDesign.zh.md](https://metanet/4-TestDesign.zh.md)

交叉引用索引

测试 ID → 设计章节 → 代码文件 的快速查找表。

测试类别	设计章节	代码路径
T1 Protobuf Schema	四 (Metanet 交易格式)	src/proto/bitfs_test.go
T2 HD Wallet	二-B (HD 钱包派生规则详细设计)	src/internal/method42/hdwallet_test.go
T3 Method 42 加密		

测试类别	设计章节	代码路径
	五 (数据类型与加密模型), 二-B.D (Method 42 加密密钥派生)	src/internal/method42/encrypt_test.go
T4 Vault 管理	二 (HD 钱包与 Vault)	src/internal/method42/vault_test.go
T5 握手协议	十三-B.B (Method 42 握手协议)	src/internal/method42/handshake_test.go
T6 Key Capsule & HTLC	十三-B.D (HTLC 购买协议)	src/internal/method42/keycapsule_test.go
T7 Key Cache	十一 (买卖交易)	src/internal/method42/keycache_test.go
T8 Metanet 节点与构建器	四-B (Metanet 交易结构详细设计)	src/internal/metanet/metanet_test.go
T9 文件系统操作	四-B (14 种文件系统操作), 三 (Unix 文件系统模型)	src/internal/metanet/fs_test.go
T10 Metanet 解析器	四-B (Metanet 交易结构详细设计)	src/internal/metanet/metanet_test.go
T11 内容寻址存储	七 (SPV 模式)	src/internal/storage/store_test.go
T12 SPV 客户端	七 (SPV 模式)	src/internal/spv/spv_test.go
T13 DNSLink	六 (DNSLink 与发布)	src/internal/dnslink/dnslink_test.go
T14 URI 寻址	六 (DNSLink 与发布)	src/internal/addressing/uri_test.go
T15 Daemon HTTP	十三-B.A (HTTP API 详细规范)	src/internal/daemon/daemon_test.go
T16 CLI 工具	八 (b* 独立工具), 九-B (CLI 命令详细参考)	src/cmd/*_test.go
T17 CLI 通用	八 (b* 独立工具)	

测试类别	设计章节	代码路径
		src/internal/cli/ common_test.go
T18 配置与错误处理	十六 (错误处理)	src/internal/config/ config_test.go
T19 Shell 交互	十 (Shell 交互模式), 九-B.C (Shell 命令参考)	src/internal/shell/ shell_test.go
T20 集成/端到端	全部章节	src/internal/ integration/e2e_test.go
T21 属性不变量	多处 (见 T21 各条目的设计参 照)	src/internal/ integration/ property_test.go
T22 安全性	五, 十一, 十三-B	src/internal/ integration/ security_test.go
T23 会话管理 (Lock/Unlock)	二十一 (会话管理)	src/internal/method42/ session_test.go
T24 权限管理 (ACL + 群签名)	二十二 (权限管理)	src/internal/acl/ acl_test.go
T25 收益权/ISO	十一-f (收益权表与 ISO)	src/internal/revenue/ iso_test.go
T26 同步与批量发布 (bsync/ bput)	二十三 (bsync/bput)	src/internal/sync/ bsync_test.go
T27 Paymail 集成	六 (DNSLink/Paymail), 十六-B (Paymail 详细设计)	src/internal/paymail/ paymail_test.go